

Pytest Agent Project Report

Jeremy Carleton

December 2025

1 Summary

Pytest Agent is a project which aims to automatically generate a suite of test cases for the purposes of regression testing through collaboration between a highly capable LLM which is expensive to query and a smaller LLM which is cheap to query. Since the target application is regression testing, the focus is on generating tests which conform to the behavior of the existing code (which is assumed to be correct) while maximizing the code coverage. The collaboration consists of the large LLM (called the planner) creating a list of concise plans for test cases in a single query and the small LLM (called the coder) generating a complete test case to fulfill each plan, using guidance from the large LLM as context. Since the small LLM is cheap to query, it is given multiple attempts to generate a correct test (i.e. one which the given code passes) and takes in feedback to refine its code each time it fails. This project focuses on python code and uses the pytest¹ framework. Evaluations on small-sized open-source projects validate Pytest Agent's potential to achieve high code coverage but indicate significant room for improvement in how the coder refines its test cases.

2 Demo Video

A video demonstration of how to run Pytest Agent on a simple python codebase is available [here](#).

3 Implementation Details

3.1 Plan Generation

The first step in the test case generation process is providing all relevant code from the project to the planner LLM so that it can generate a set of test plans for the coder LLM to handle individually. The relevant files (i.e. those which define the functions and classes to be tested) are identified, and the contents of those files are provided as context to the planner LLM. All of this code is

¹<https://github.com/pytest-dev/pytest>

appended to a prompt which explains the role of the LLM, what details to provide for each test plan, and how to format the test plan. We choose to have the planner LLM output its plans in the JSON format since this makes it very easy to convert the output text into a Python dictionary. Each entry in the JSON object uses the test case name as its key, and has the following fields: **functions_required** (a list of functions which are expected to be needed for implementing the test), **classes_required** (a list of classes which are expected to be needed for implementing the test), and **description** (a description of how to setup, what conditions to check, and how to teardown if needed).

The planner LLM is responsible for understanding a highly complex task and breaking it down for the coder LLM, so a highly capable model should be used. We used OpenAI’s GPT-4.1 model, accessed through TAMU AI Chat via the `llm api`². The model could alternatively be accessed through OpenAI directly using one of their API keys with minimal changes to the code.

3.2 Test Code Generation

The prompts used by the coder LLM can be found in `src/ut/prompts/`. There are three prompts, one which is used when no feedback from prior attempts is available, one which is used when the previous attempt failed the linter check, and one which is used when the previous attempt was run but resulted in a failure. Here we focus on the first prompt, and in section 3.4 we discuss the other two. The prompt begins with a line explaining the role of the LLM, followed by a summary of its task. Then, an example of the desired output format is given. This is followed by additional instructions, such as an example of how to test for an exception being raised. Finally, the code which the planning LLM identified as relevant to the test is provided. Since we want the coder to use the imports recommended by the plan, we forcefully insert them into the generated code.

We use vLLM as the inference server for the coder LLM. To accelerate the generation of test code, a separate server is started on each available GPU and the set of test generation prompts is distributed between them. The system was tested on a machine with four 96GB H100 GPUs, so each could fit the Qwen2.5-Coder-32B-Instruct model. In this way, multiple tests are generated in parallel. Once all tests have been generated, we extract the imports and the test function from between the `“python` and `“` markers if they are present. Each test is evaluated using the `ruff`³ linter before being added to the full test file, as explained in the next section.

3.3 Test Code Evaluation

Every test case generated by the coder LLM is evaluated using the `ruff` linter to screen for any obvious syntax errors that would prevent `pytest` from running properly. Since this step is only intended to ensure that all tests are runnable,

²<https://github.com/simonw/llm>

³<https://github.com/astral-sh/ruff>

we configure the linter to ignore all stylistic rules or unused variable errors. Furthermore, since the linter is not aware of code which the test will be run on, we do not check for import errors. When a syntax error is caught by the linter, the description of the syntax error is saved for use as feedback to the coder LLM.

All tests which pass the linter check are compiled into a single python file, will all the relevant imports (both those used to import the code under test and those added by the coder LLM) placed at the top. This file is copied into a Docker container which already contains the project code, and pytest is run with the `pytest-cov`⁴ plugin on that file. The terminal output from pytest and the coverage report file (if one was generated) are extracted from the container; the pytest output is parsed immediately while the coverage report is saved locally. Parsing the pytest output tells us (a) whether the tests were able to run, (b) which tests passed and which tests failed (if they were able to run), and (c) summaries of any errors or causes of test failures. If the tests were able to run, we take the tests which passed and save their code to a temporary file so it can be added to the final test file at the end. For the tests which failed, we take the error summaries for each one from the pytest output and save them for use as feedback to the coder LLM. Sometimes, the tests are not able to run due to an error which is not caught by the linter. In these cases, we further parse the error message from pytest and use a rule-based method to determine whether the cause was a syntax error inside a function or an import error. We then attempt to remove the source of the error from the file before copying it and running pytest again. Any test cases which have to be removed as a result of this are marked as failed. The process can be repeated multiple times in cases where there are multiple errors stopping pytest from running. At the end, if the tests still could not be run we mark them all as failed.

3.4 Iterative Refinement

Once testing is complete, the tests which have not yet passed can be divided into three categories: those for which no valid python function definition was parsed from the output of the coder LLM, those which failed to pass the linter test, and those which failed when pytest was run. The prompt used when revising the test with the coder LLM is different in each of these cases. In the first case, there is no clear feedback to provide so the same prompt is used again. Using a temperature of 0.2 when generating with the coder LLM gives a strong likelihood that the output will be different from the previous attempt, so it is possible for the LLM to make progress. In the second case, the code from the LLM's previous attempt is included in the prompt along with the description of the syntax error returned by the linter. The third case is similar except the error description comes from pytest.

The iterative refinement process consists of repeatedly prompting the coder LLM with feedback in this way and using the same process described in sec-

⁴<https://github.com/pytest-dev/pytest-cov>

tion 3.3 to re-evaluate and collect new feedback. As new passing tests are generated, those tests are removed from the list that need to be generated in the next iteration. Since this process only requires the small locally-hosted coder LLM, it is relatively inexpensive to perform many iterations; however, we currently find that the number of passing tests saturates after only a few iterations. Once the desired number of iterations is completed, any tests which have passed are compiled into a single python file which can be used for regression testing.

4 Development Process

This project is a fork of an existing open-source project called unittest-ai-agent⁵. The goal of that project is the same as Pytest Agent, but it relies entirely on the GPT-4o model which writes the test cases directly. The GPT-4o model is prompted for each function to be tested individually, potentially resulting in several API calls. Furthermore, the scope of unittest-ai-agent is limited to just writing the tests; all evaluation must be done manually. Development began with implementing an automated system for running the generated tests, then moved on to creating a framework for collaboration between the planner LLM and the coder LLM, and finally focused on implementing a refinement procedure for the generated tests. This process required a major overhaul of the codebase. A full diff report of the changes can be found [here](#).

5 Evaluation Results

Our objectives in evaluating Pytest Agent are to see how good of a job the coder LLM does at generating valid test cases using the planner’s instructions and the various forms of feedback, and what kind of coverage over the code under test this achieves. We initially selected the following ten open-source projects to evaluate Pytest Agent on (click each name to be taken to the Github repo):

- [docopt](#)
- [parse](#)
- [portion](#)
- [purl](#)
- [PathSpec](#)
- [srt](#)
- [Tomli](#)
- [untangle](#)

⁵<https://github.com/herchila/unitest-ai-agent>

- writemplate
- url-normalize

Note that we could not include the docopt project in the evaluation results due to issues getting pytest to run, and Pytest Agent failed to generate any passing tests for the python-pathspect project. For the rest of the projects, we present the test pass rates in fig. 1, fig. 2, and fig. 3. We also show the final code coverage statistics in table 1. Note that in all cases the coder LLM was given three attempts (that is, three iterations).

Overall, the pass rate results show that improvement in the number of passing tests diminishes very quickly after the first two iterations even if there is still a large number of remaining tests. This shows the need for better ways to help the coder LLM revise its code. Since the goal is to avoid incurring the costs of querying the large planner LLM many times, it cannot be used in this process. Thus, the coder LLM must interpret any feedback on its own; this makes achieving significant improvement after the first iteration challenging. Fortunately, table 1 shows that even with many planned tests not being included in the final file, the final code coverage is often above 80%.

6 Future Work

As alluded to in section 5, the most important direction for future work is improving the capability of the small coder LLM to correct its test cases. An obvious way to do this is to provide it with additional information coming from the project documentation (if available) that relates to the test plan and the feedback it has received. Since the model may be too small or ignorant to use search tools directly, a RAG system may be the best way to go about retrieving that information.

Another direction for future work is to have the planner LLM create a new plan based on the code coverage results once the coder LLM has produced passing tests for most or all of its original plans. The coverage reports provided by pytest-cov break down the level of coverage on a function-by-function level, so it would be easy to see which functions need more attention.

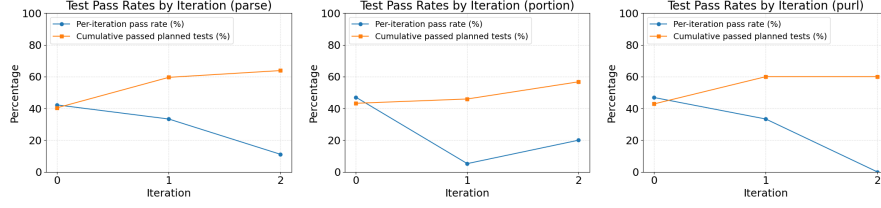


Figure 1: Test case pass rates for (a) parse, (b) portion, (c) purl. The blue curve indicates what percentage of the tests evaluated on a given iteration passed, and the orange curve indicates what percentage of the total number of planned tests had passed up to the given iteration.

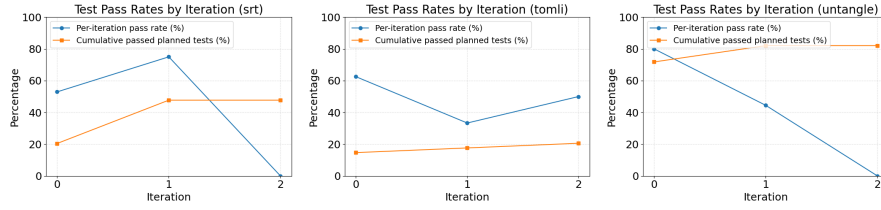


Figure 2: Test case pass rates for (a) srt, (b) Tomli, (c) untangle. The blue curve indicates what percentage of the tests evaluated on a given iteration passed, and the orange curve indicates what percentage of the total number of planned tests had passed up to the given iteration.

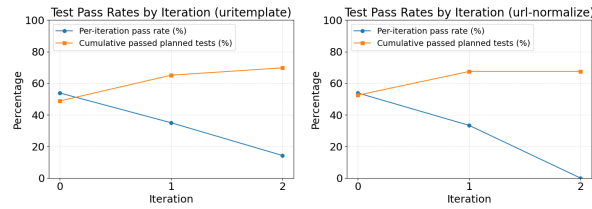


Figure 3: Test case pass rates for (a) uritemplate, (b) url-normalize. The blue curve indicates what percentage of the tests evaluated on a given iteration passed, and the orange curve indicates what percentage of the total number of planned tests had passed up to the given iteration.

Project Name	# Files	# Passed Tests	Total Lines Covered	% Coverage
parse	1	30	647	75.94
portion	6	21	788	74.90
purl	2	21	457	83.55
srt	1	21	257	87.12
Tomli	2	7	315	56.76
untangle	1	32	348	96.94
writemplate	4	30	484	85.66
url-normalize	13	27	303	97.43

Table 1: Coverage statistics from the evaluation. For each project, the number of files containing code under test is listed (which excludes any `__init__.py` files), along with the total number of lines covered and the percentage of all lines covered.